

AgriCoop Marketplace

Complete development, architecture, workflows, and operations
documentation

AgriCoop Development Team \ Technical handoff and defense-demo reference \ 2026-10-07

Table of Contents

1	Executive summary	1
2	Product purpose and problem statement	1
2.1	Purpose	1
2.2	Intended users	1
3	Development process	2
3.1	Discovery and requirements extraction	2
3.2	Engineering principles used	2
4	Current technology stack	2
5	System architecture	3
5.1	Current runtime architecture	3
5.2	Native migration architecture	4
6	Data model and persistence	4
6.1	Main entities	4
6.2	Database migrations	4
7	Authentication and authorization	4
7.1	Registration and login flow	4
7.2	Authorization layers	5
7.3	Role matrix	5
8	Public user experience	6
8.1	Landing page	6
8.2	Marketplace browsing	6
9	Seller workflow	6
9.1	Product listing lifecycle	6
9.2	Seller order and quotation activity	7
10	Buyer workflow	7
10.1	Discover and compare	7
10.2	Quotation request	7
10.3	Order flow	7
11	Payment and transaction policy	7
12	Conversations and real-time support	8
12.1	Why conversations exist	8
12.2	Access control	8
12.3	Real-time mechanism	8
13	Administration, moderation, and reporting	8
13.1	Admin dashboard	8
13.2	Product moderation queue	8
13.3	Reports	8
14	Demo data and defense preparation	8
15	Security and privacy considerations	9
15.1	Secrets	9

15.2	Data protection	9
15.3	Known security boundary	9
16	Testing and quality assurance	9
17	Deployment and operations	10
17.1	Current runtime deployment	10
17.2	Native Next.js path	10
17.3	RLS application procedure	11
18	File and directory guide	11
19	End-to-end scenario walkthroughs	12
19.1	Buyer purchases a listing	12
19.2	Buyer requests a quotation	12
19.3	Seller submits a product	12
19.4	Admin monitors the cooperative	12
20	Limitations and future roadmap	12
21	Glossary	13
22	Conclusion	13

1 Executive summary

AgriCoop is a cooperative-centered agricultural marketplace designed for local agricultural trade in Butuan City, Agusan del Norte. The platform connects Buyers and Sellers/Farmers, supports quotations and regular or bulk orders, records face-to-face payment coordination, and provides administration tools for product moderation, member oversight, reports, and support conversations.

The system was developed as a thesis-oriented prototype and evolved through several implementation stages. The current verified defense-demo runtime is a React 19 and Vite client served by an Express 4 server. tRPC provides the typed API contract; Drizzle ORM maps the business schema to MySQL/TiDB; Supabase Auth verifies bearer sessions; and an authenticated Server-Sent Events broker provides immediate conversation updates. A separate native Next.js/Tailwind/Vercel shell and a Supabase/Postgres RLS migration artifact are also included as the migration path.

Important scope statement. The current production-like runtime remains Vite + Express + Drizzle/MySQL. The native Next.js shell builds independently, but the full business-router migration to Supabase Postgres and live RLS application require a Postgres connection and further porting work.

2 Product purpose and problem statement

2.1 Purpose

AgriCoop exists to make cooperative agricultural trading easier to discover, coordinate, document, and supervise without removing the face-to-face relationships that are important to local cooperatives. It is not simply an online store. It is a coordination system for:

- making cooperative produce visible;
- comparing products, prices, units, quality grades, and available stock;
- requesting formal quotations before committing to volume purchases;
- placing regular and multi-product bulk orders;
- coordinating F2F payment and handover arrangements;
- giving Sellers a controlled listing workflow;
- giving Admins an operational view of members, products, orders, and reports; and
- preserving a written conversation trail for inquiries, quotations, and orders.

2.2 Intended users

The final application model uses four roles:

Role	Purpose and capabilities
Buyer	Browse approved listings, request quotations, place regular or bulk orders, review activity, coordinate F2F handover, and message Sellers.
Seller / Farmer	Create and update listings, maintain inventory, submit products for verification, respond to quotations, fulfill orders, and message Buyers.
Admin	Manage cooperative members, monitor operations, review and approve or reject Seller products, export reports, and oversee support threads.

Role	Purpose and capabilities
SuperAdmin	Platform-level administrative authority. SuperAdmin is not available through public registration and has precedence over normal Admin controls.

The original thesis language referred to cooperative officers and system administrators. In the current implementation, cooperative oversight is represented by Admin procedures and views, while SuperAdmin represents platform-wide administration. This consolidation avoids an unsafe public officer-registration path while preserving the required monitoring and reporting functions.

3 Development process

3.1 Discovery and requirements extraction

The project began by scanning the thesis manuscript and translating its goals into implementable product requirements. The requirements were grouped into public browsing, authentication, role workspaces, marketplace rules, quotations, ordering, payments, inventory, reporting, and deployment.

The development process then followed an incremental preservation strategy:

1. Preserve the existing landing page, marketplace browse experience, dashboards, responsive layout, and CSV reporting.
2. Add the business tables and procedures required for quotations, orders, inventory, transactions, and conversations.
3. Add custom Supabase Auth login and registration while retaining the typed server contract.
4. Add the F2F-only payment policy requested for the simplified final scope.
5. Add Seller verification and Admin moderation so unapproved products cannot appear in the public marketplace.
6. Add support conversations linked to quotation and order events.
7. Add authenticated SSE updates for connected conversation participants.
8. Add deterministic tests, seed data, environment documentation, and migration artifacts.
9. Add a native Next.js/Tailwind shell and document the remaining Postgres cutover boundary.

3.2 Engineering principles used

The implementation emphasizes typed boundaries, server-side authorization, data validation, non-destructive migrations, clear UI states, and reproducible defense-demo data. Critical operations use protected tRPC procedures rather than relying on hidden buttons. Public pages can browse only products that pass the database-level marketplace eligibility filters.

4 Current technology stack

Layer	Technology and responsibility
Client	React 19, Vite 7, Wouter routing, Tailwind CSS 4, shadcn-style Radix UI components, Lucide icons, React Query, and tRPC React bindings.

Layer	Technology and responsibility
Server	Express 4, tRPC 11, TypeScript, server-sent events for conversation updates, and storage proxy support.
Data access	Drizzle ORM with MySQL/TiDB in the current runtime. The schema is defined in <code>drizzle/schema.ts</code> .
Authentication	Supabase Auth email/password sessions. The browser forwards the active access token as a Bearer token; the server verifies it with the Supabase admin client.
Authorization	Server-side role checks in <code>server/authz.ts</code> , protected tRPC procedures, ownership checks, product moderation checks, and a target Supabase RLS artifact.
Testing	Vitest authorization, marketplace-rule, CSV-report, credential, and logout tests; TypeScript checking; Vite/esbuild production build; native Next.js build and route smoke tests.
Migration path	Next.js 16 app router shell, Tailwind v4 PostCSS integration, <code>next.config.mjs</code> , Supabase CLI configuration, and <code>supabase/migrations/20261005_rls_baseline.sql</code> .

5 System architecture

5.1 Current runtime architecture

The current runtime is a single Node process that serves the Vite client in development or built static assets in production, exposes tRPC under `/api/trpc`, and exposes the authenticated conversation stream under `/api/conversations/:conversationId/stream`.

```

Browser (React + Wouter + React Query)
  |
  | Supabase access token in Authorization header
  v
Express server
  |-- /api/trpc -> typed tRPC procedures -> authz -> Drizzle/MySQL
  |-- /api/conversations/:id/stream -> Supabase token check -> SSE broker
  |-- storage proxy
  `-- Vite dev server or static production assets
      |
      v
Supabase Auth (identity verification only in current runtime)

```

5.2 Native migration architecture

The repository also contains a separate `app/` directory for a native Next.js route tree. It currently provides a polished landing shell and working static routes for `/`, `/about`, `/marketplace`, `/support`, and `/auth`. Tailwind v4 is wired through `postcss.config.mjs`; `next:build` and `next:start` are available for Vercel-style deployment validation.

The native shell is intentionally not described as a completed business migration. Completing that migration requires porting the tRPC procedures and Drizzle/MySQL schema to Next.js route handlers or server actions backed by Supabase Postgres, then applying and testing RLS.

6 Data model and persistence

6.1 Main entities

The primary schema contains the following business concepts:

- `users`: identity-linked local profile, role, approval status, cooperative assignment, contact details, and sign-in metadata;
- `cooperatives`: cooperative name, location, description, and status;
- `products`: Seller ownership, cooperative, name, category, unit, price, stock, quality grade, description, publication status, verification status, visibility, rejection reason, and image metadata;
- `quotations`: product, Buyer, Seller, requested quantity, notes, quoted price, response note, status, and response time;
- `orders`: Buyer, Seller, cooperative, order type, status, F2F payment state, total, notes, and completion time;
- `order_items`: product-level quantities and unit prices for regular and bulk orders;
- `transactions`: order, Buyer, Seller, amount, F2F payment method, transaction status, and handover/reference notes;
- `conversations`: Buyer/Seller participants, product, order or quotation context, subject, status, and timestamps; and
- `conversation_messages`: sender, conversation, message body, and timestamps.

6.2 Database migrations

Schema changes are generated with Drizzle Kit and stored as SQL under `drizzle/`. The current migration history includes the original application tables, conversation tables, role/product moderation changes, and participant fields. Role changes are data-safe: legacy values are normalized before narrowing the role enum.

The Supabase/Postgres target artifact is separate under `supabase/rls.sql` and `supabase/migrations/20261005_rls_baseline.sql`. It assumes the Postgres profile table has an `auth_user_id` UUID that points to `auth.users(id)`.

7 Authentication and authorization

7.1 Registration and login flow

1. A visitor opens `/auth`.

2. The user chooses sign in or registration.
3. Public registration accepts only buyer or seller.
4. The server-side registration procedure creates a confirmed Supabase Auth user with `email_confirm: true`, stores role metadata, and creates the local profile.
5. The browser signs in with Supabase email/password authentication.
6. Supabase emits a session containing an access token.
7. `main.tsx` tracks the session and attaches the token to tRPC requests.
8. `server/_core/context.ts` verifies the token with Supabase, loads or creates the local profile, and exposes the role-aware user in the tRPC context.
9. Logout calls the app logout mutation and `supabase.auth.signOut()`, clears the local query cache, and invalidates protected data.

Public registration cannot create Admin or SuperAdmin accounts. Privileged roles are assigned through administrative controls. Existing server-assigned Admin/SuperAdmin roles are preserved when a Supabase session is refreshed.

7.2 Authorization layers

Authorization is intentionally repeated at several boundaries:

- client navigation and role-aware views improve usability;
- protected tRPC procedures reject unauthenticated requests;
- `requireOperationalRole` checks the allowed role set;
- ownership checks prevent users from editing another Seller's products or reading another participant's conversation;
- product purchase eligibility requires published, approved, visible products;
- self-transactions are blocked when Buyer/Seller ownership would be the same; and
- the target Postgres migration adds database-enforced RLS policies after the future cutover.

7.3 Role matrix

Area	Buyer	Seller	Admin	SuperAdmin
Marketplace browsing	Yes	Yes	Yes	Yes
Create listing	No	Yes	No	No
Submit product for review	No	Yes	No	No
Approve/reject product	No	No	Yes	Yes
Place order	Yes	Yes if eligible	No	No
Request quotation	Yes	Yes if eligible	No	No
Manage cooperative members	No	No	Yes	Yes

Area	Buyer	Seller	Admin	SuperAdmin
Platform controls	user	No	Limited	Yes
Conversation oversight	Participant	Participant	All	All

8 Public user experience

8.1 Landing page

The landing page uses a farm-fresh visual system: deep cooperative green, muted harvest green, warm off-white surfaces, rounded cards, serif display headings, and direct calls to action. Its purpose is to explain the value proposition before asking users to authenticate.

The main public routes are:

- /: product story and entry points;
- /about: How AgriCoop works and role explanation;
- /marketplace: searchable product discovery;
- /auth: Supabase sign in and registration;
- /support: inquiries and conversation entry point;
- /activity: Buyer/Seller outgoing orders and quotations; and
- /reports: report-oriented views for authorized users.

8.2 Marketplace browsing

The marketplace query filters by optional category and sort order. Public results are restricted to products whose status is published, verification status is approved, and visibility is visible. The UI supports search/filtering, category selection, price sorting, stock display, quality grade, Seller information, cart preparation, quotation requests, and F2F-only checkout coordination.

9 Seller workflow

9.1 Product listing lifecycle

1. Seller opens the dashboard and selects the listing workflow.
2. Seller enters product name, category, description, unit, price, stock quantity, quality grade, and up to ten image URLs.
3. A new record begins as a draft or pending verification according to the submission choice.
4. Seller can update the product and adjust inventory.
5. Seller submits the product for verification.
6. Admin reviews the pending queue.
7. Admin approves or rejects the listing. A rejection reason is stored for correction and resubmission.
8. Only approved and visible listings appear in the public marketplace.

This process protects marketplace quality and makes moderation explicit rather than relying on hidden UI controls.

9.2 Seller order and quotation activity

Sellers can participate as Buyers when the business rule permits it, which supports seller-to-seller and cooperative-to-cooperative trade. The system blocks self-purchase and self-quotation attempts. Seller activity includes outgoing quotations, orders, inventory changes, order fulfillment status, and support messages.

10 Buyer workflow

10.1 Discover and compare

A Buyer browses approved listings, filters by category, sorts by price, and adds eligible products to the cart. Bulk orders are normalized to a single Seller to keep fulfillment and handover coordination clear.

10.2 Quotation request

A Buyer selects a product, enters a requested quantity, and adds an optional note. The server validates product ownership, availability, and self-transaction rules before creating the quotation. A linked support conversation is opened automatically so the Buyer and Seller can clarify pricing, quantity, availability, and F2F arrangements.

The quotation lifecycle supports requested and responded states, quoted price, Seller response note, and response timestamp. Quotations can be monitored in authorized dashboards and reports.

10.3 Order flow

1. Buyer adds one or more products from the same Seller.
2. Client calculates a preview total; the server recalculates from authoritative product prices.
3. Server verifies stock, product approval/visibility, role eligibility, and self-transaction protection.
4. Order and order items are created.
5. Inventory is decremented as part of the order operation.
6. The order receives a fulfillment status and F2F payment coordination status.
7. A linked support conversation is created with a handover prompt.
8. Buyer and Seller coordinate the physical exchange and payment.
9. Seller/Admin records completion and reference notes.

11 Payment and transaction policy

The final simplified scope is deliberately F2F-only. Online payment gateway code, PayMongo intents, QR flows, and webhooks were removed from the active application path to avoid unnecessary complexity for the defense demo.

An order records:

- `paymentMethod: f2f;`
- a payment/coordination status such as pending, paid, or F2F-pending-confirmation;
- a transaction row when the payment record is documented; and
- a reference note such as “Cash handed over at cooperative office.”

This is a coordination record, not an online payment processor. The product language consistently explains that payment and handover occur face-to-face.

12 Conversations and real-time support

12.1 Why conversations exist

Conversations keep support, quotations, and handover coordination in one place. They reduce ambiguity without adding a complex chat platform. A direct inquiry creates a general thread. A quotation request creates a quotation-linked thread. An order creates an order-linked thread with a prompt to confirm availability, F2F payment, and handover details.

12.2 Access control

Buyers and Sellers can access their own participant threads. Admin and SuperAdmin can oversee threads. Server procedures validate the conversation participant IDs before returning messages or accepting replies. The restored router allows both Buyer and Seller participants to read and respond.

12.3 Real-time mechanism

The current runtime uses a lightweight in-process event broker in `server/realtime.ts`. The Express endpoint authenticates the Supabase bearer token, verifies the local user, verifies conversation membership, then keeps an SSE connection open. New messages publish an event to listeners for the same conversation. The Support page refetches message data when a message event arrives.

This is suitable for a single-process defense demo. A horizontally scaled deployment should replace the in-memory broker with Supabase Realtime, Redis pub/sub, or another hosted event system.

13 Administration, moderation, and reporting

13.1 Admin dashboard

The Admin workspace contains member validation, cooperative monitoring, inventory pulse, product moderation, reports, user role/approval management, and support oversight. SuperAdmin enters the administrative workspace with platform-level access.

13.2 Product moderation queue

Admin sees pending product submissions and can approve or reject them. Approval sets verification status to approved, visibility to visible, and publication status to published. Rejection sets verification status to rejected, visibility to hidden, archives the product, and stores an optional rejection reason.

13.3 Reports

CSV reporting covers sales, inventory, quotations, and transactions. Serialization is centralized and tested to ensure predictable headers, escaping, and row output. Reports are intended for operational review and defense-demo evidence rather than accounting replacement.

14 Demo data and defense preparation

The `scripts/seed-demo.ts` script is designed for a non-production demonstration database. It creates or reuses:

- one cooperative named for the demo environment;
- three Seller/Farmer demo profiles;

- one Buyer demo profile;
- an Admin demo profile;
- twelve products across fruits, grains, root crops, vegetables, and legumes;
- approved and visible product listings with stock and quality grades;
- requested and responded quotations;
- completed and active order examples;
- F2F transaction records with reference notes; and
- a linked support conversation for order handover.

Never run the seed command against a production database. The script is designed to be idempotent for the intended demo records, but it is still a data-writing operation and should be isolated to a defense or staging database.

Useful command:

```
pnpm seed:demo
```

15 Security and privacy considerations

15.1 Secrets

Browser-safe variables are `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Server-only secrets include `SUPABASE_SERVICE_ROLE_KEY`, `DATABASE_URL`, and `JWT_SECRET`. Secret values must stay in the deployment environment and must never be committed to Git or pasted into public documentation.

15.2 Data protection

The current runtime uses server-side authorization and MySQL constraints/queries. The future Supabase target adds RLS policies for users, products, quotations, orders, transactions, conversations, and messages. The RLS functions derive the application role and cooperative from the authenticated user profile.

15.3 Known security boundary

The RLS file is a target artifact, not proof that the current MySQL runtime has Postgres RLS. It assumes a Postgres schema with `auth_user_id` profile links. Live application requires a `SUPABASE_DB_URL` or equivalent direct database connection. The current documented environment did not provide one.

16 Testing and quality assurance

The verified deterministic suite contains six Vitest files and twelve tests:

Test file	Count	Coverage
auth.logout.test.ts	1	Logout behavior and user fixture compatibility.

Test file	Count	Coverage
authz.test.ts	2	Buyer, Seller, Admin, and SuperAdmin authorization boundaries.
credentials.test.ts	1	Optional credential configuration behavior.
marketplaceRules.test.ts	5	Quotation state, order normalization, stock, transaction status, and fulfillment transitions.
reportCsv.test.ts	2	CSV serialization and escaping.
serverCredentials.test.ts	1	Network-independent server credential checks.

The standard verification commands are:

```
pnpm check
pnpm test
pnpm build
pnpm run next:build
```

The native Next shell was also started with `next start` on a test port and smoke-tested at `/`, `/about`, `/marketplace`, `/support`, and `/auth`; each returned HTTP 200.

17 Deployment and operations

17.1 Current runtime deployment

The current defense-demo application can be run locally with:

```
pnpm install
# create .env using docs/ENVIRONMENT.md
pnpm dev
```

For a production-like build:

```
pnpm build
pnpm start
```

The runtime expects a MySQL/TiDB `DATABASE_URL`, Supabase public Auth values, and a server Supabase service role key. It is not yet a standalone Next.js + Supabase Postgres deployment.

17.2 Native Next.js path

The repository includes:

- `app/layout.tsx` and `app/page.tsx`;
- native routes under `app/about`, `app/marketplace`, `app/support`, and `app/auth`;
- `next.config.mjs`;
- `postcss.config.mjs` with Tailwind v4; and
- `next:dev`, `next:build`, and `next:start` scripts.

These files establish a deployable Next.js shell. The remaining migration work is to connect these routes to Supabase Auth, port business procedures to Postgres-backed server actions or route handlers, replace the MySQL schema, and move realtime support to a hosted service.

17.3 RLS application procedure

After provisioning the target Supabase project and obtaining a direct Postgres connection privately:

```
export SUPABASE_DB_URL='postgresql://...'
supabase db push --db-url "$SUPABASE_DB_URL"
```

Then verify with authenticated Buyer, Seller, Admin, and SuperAdmin sessions. Confirm that an unapproved product is not visible, a Buyer cannot read another Buyer's order, a Seller cannot edit another Seller's product, and Admin/SuperAdmin oversight works as intended.

18 File and directory guide

Path	Responsibility
<code>client/src/pages/</code>	Public pages, marketplace, dashboards, activity, reports, support, and auth.
<code>client/src/components/AgriShell.tsx</code>	Public/workspace shell, navigation, and shared page framing.
<code>client/src/main.tsx</code>	React bootstrap, Supabase session forwarding, React Query, and tRPC client.
<code>server/routers.ts</code>	Typed procedures for auth, marketplace, products, quotations, orders, conversations, reports, and admin controls.
<code>server/_core/context.ts</code>	Supabase bearer verification and local role-aware context construction.
<code>server/db.ts</code>	Drizzle query helpers, marketplace filters, conversations, reports, and upserts.
<code>server/authz.ts</code>	Role and ownership authorization helpers.
<code>server/realtime.ts</code>	In-process conversation event broker for SSE.
<code>drizzle/schema.ts</code>	Current MySQL/TiDB schema definitions.
<code>drizzle/*.sql</code>	Generated and reviewed database migrations.

Path	Responsibility
scripts/seed-demo.ts	Defense-demo cooperative, accounts, products, quotations, orders, transactions, and conversation data.
supabase/rls.sql	Target Postgres RLS policy artifact.
supabase/migrations/	Supabase CLI migration wrapper for the target Postgres database.
app/	Native Next.js deployment shell and routes.
docs/ENVIRONMENT.md	Environment variable and migration guidance.

19 End-to-end scenario walkthroughs

19.1 Buyer purchases a listing

Buyer signs in, opens the marketplace, filters to a product category, adds a product, confirms quantity, and submits an F2F order. The server recalculates the total, verifies stock and eligibility, creates order items, decrements inventory, records F2F coordination, and opens a support thread. Buyer and Seller then use the thread to confirm pickup and handover. Seller or Admin later records the transaction note and completion status.

19.2 Buyer requests a quotation

Buyer chooses a product, enters a requested quantity and note, and submits. The server verifies the product and ownership rules, creates a quotation in the requested state, and opens a quotation-linked conversation. Seller reviews the request, responds with a price and note, and the Buyer sees the response in activity and support views.

19.3 Seller submits a product

Seller saves product details and submits the listing for verification. Admin sees the pending listing, checks product information, and approves or rejects it. A rejected listing remains hidden and retains the rejection reason. After correction and resubmission, approval makes it eligible for public marketplace discovery.

19.4 Admin monitors the cooperative

Admin opens the dashboard to review member status, inventory pulse, quotations, orders, transactions, pending product submissions, and report snapshots. Admin can export CSV reports and oversee support conversations. SuperAdmin can perform platform-wide role and cooperative controls.

20 Limitations and future roadmap

The following items are intentionally identified rather than hidden:

1. The current business runtime is MySQL/TiDB through Drizzle, not Supabase Postgres.
2. The RLS file is ready but has not been applied to a live Postgres target because no connection URL was provided.
3. The native Next.js app is a deployment shell; the full marketplace business procedures still run in the existing Express/tRPC runtime.

4. The in-process SSE broker is appropriate for one process but should be replaced for horizontal scaling.
5. Product images are represented by URL metadata and require a production storage policy and upload UX.
6. F2F payment records document coordination and handover; they do not process funds online.
7. Demo seed data should remain isolated from production data.

Recommended roadmap:

- Port `drizzle/schema.ts` to Supabase Postgres and add stable `auth_user_id` profile links.
- Convert tRPC procedures into Next.js route handlers/server actions or retain a separately deployed typed API.
- Apply and integration-test RLS with real authenticated sessions.
- Replace the in-process event broker with Supabase Realtime.
- Add production storage for product images and audit logging for Admin actions.
- Add Playwright end-to-end tests for Buyer, Seller, Admin, and SuperAdmin walkthroughs.

21 Glossary

Term	Meaning in AgriCoop
F2F	Face-to-face payment and handover coordination.
RLS	Row Level Security, a database policy mechanism planned for Supabase Postgres.
tRPC	Typed client/server API contract used by the current Express runtime.
Drizzle	TypeScript ORM used for current MySQL/TiDB persistence.
SSE	Server-Sent Events, the one-way event stream used for immediate support updates.
Verification	Admin review state for Seller product submissions.
Visibility	Whether an approved product is allowed to appear in the marketplace.
Quotation	A request for price and availability before an order commitment.
Bulk order	An order containing multiple products from one Seller with a combined total.

22 Conclusion

AgriCoop is a complete thesis-ready cooperative marketplace prototype with public browsing, custom Supabase Auth, four role-aware workspaces, Seller moderation, quotation and order workflows, inventory rules, F2F transaction records, CSV reporting, support conversations, realtime updates, seed data, tests, and a documented migration path toward native Next.js and Supabase Postgres.

Its most important design decision is to preserve cooperative coordination rather than treat the platform as a generic checkout system. Product approval protects trust, server-side authorization protects ownership, support threads preserve context, and F2F records keep the payment model aligned with the simplified defense-demo requirement.